Name - Arayana khare
Sap id- 590042432
Batch - 15
1. Algorithm
Main Program
1. Start.
2. Display the menu:
◦ Left distributive property
◦ Right distributive property
◦ Exit
3. Read the user's choice.
4. Use `switch`:
◦ Choice 1 → call `leftDistributive()`.
◦ Choice 2 → call `rightDistributive()`.
◦ Choice 3 → terminate.
◦ Otherwise → display `"Invalid choice"`.
5. Repeat until choice 3.
6. Stop.
Left Distributive Property
Verify:
\[A(B+C)=(AB)+(AC) \]
where:
• A = `m × n`
• B = `n × p`
• C = `n × p`
• Result = `m × p`
Steps:
1. Read positive values of `m`, `n`, and `p`.
2. Declare variable-length arrays A, B, and C.
3. Read all three matrices.
4. Calculate `B + C`.
5. Calculate `A × (B+C)` → LHS.
6. Calculate `A × B`.
7. Calculate `A × C`.
8. Calculate `(A×B) + (A×C)` → RHS.
9. Display LHS and RHS.
10. Compare corresponding elements.

11. If every element is equal, display that the left distributive property is verified.

Right Distributive Property

Verify:

$$(A+B)C=(AC)+(BC)$$

where:

- A = `m × n`
- B = `m × n`
- C = `n × p`
- Result = `m × p`

Steps:

1. Read positive values of `m`, `n`, and `p`.
2. Declare variable-length arrays A, B, and C.
3. Read all matrices.
4. Calculate `A + B`.
5. Calculate `(A+B) × C` → LHS.
6. Calculate `A × C`.
7. Calculate `B × C`.
8. Calculate `(A×C) + (B×C)` → RHS.
9. Display both resulting matrices.
10. Compare corresponding elements.
11. If all elements are equal, display that the right distributive property is verified.

2. C Program

```
#include <stdio.h>
/* Function prototypes */
void readMatrix(int rows, int cols, int a[rows][cols]);
void printMatrix(int rows, int cols, int a[rows][cols]);
void addMatrix(int rows, int cols,
int a[rows][cols],
int b[rows][cols],
int result[rows][cols]);
void multiplyMatrix(int r1, int c1, int c2,
int a[r1][c1],
int b[c1][c2],
int result[r1][c2]);
int compareMatrix(int rows, int cols,
```

```
int a[rows][cols],
int b[rows][cols]);
void leftDistributive(void);
void rightDistributive(void);
int main(void)
{
int choice;
do
{
printf("\n========== HDOC DAY-34 ==========\n");
printf("1. Verify Left Distributive Property\n");
printf("2. Verify Right Distributive Property\n");
printf("3. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice)
{
case 1:
leftDistributive();
break;
case 2:
rightDistributive();
break;
case 3:
break;
printf("Exiting the program...\n");
default:
printf("Invalid choice. Please enter 1, 2, or
3.\n");
}
} while (choice != 3);
return 0;
}
```

```
/* Read matrix elements */
void readMatrix(int rows, int cols, int a[rows][cols])
{
int i, j;
for (i = 0; i < rows; i++)
{
for (j = 0; j < cols; j++)
{
scanf("%d", &a[i][j]);
}
}
}
/* Display a matrix */
void printMatrix(int rows, int cols, int a[rows][cols])
{
int i, j;
for (i = 0; i < rows; i++)
{
for (j = 0; j < cols; j++)
{
printf("%d\t", a[i][j]);
}
printf("\n");
}
}
/* Add two matrices */
void addMatrix(int rows, int cols,
int a[rows][cols],
int b[rows][cols],
int result[rows][cols])
{
int i, j;
for (i = 0; i < rows; i++)
{
for (j = 0; j < cols; j++)
{
result[i][j] = a[i][j] + b[i][j];
}
```

```
}
}
/* Multiply two matrices */
void multiplyMatrix(int r1, int c1, int c2,
int a[r1][c1],
int b[c1][c2],
int result[r1][c2])
{
int i, j, k;
for (i = 0; i < r1; i++)
{
for (j = 0; j < c2; j++)
{
result[i][j] = 0;
for (k = 0; k < c1; k++)
{
result[i][j] += a[i][k] * b[k][j];
}
}
}
}
/* Compare two matrices */
int compareMatrix(int rows, int cols,
int a[rows][cols],
int b[rows][cols])
{
int i, j;
for (i = 0; i < rows; i++)
{
for (j = 0; j < cols; j++)
{
if (a[i][j] != b[i][j])
{
return 0;
}
}
}
```

```
return 1;
}
/* Verify A(B+C) = AB + AC */
void leftDistributive(void)
{
int m, n, p;
do
{
printf("\nEnter m, n and p: ");
scanf("%d %d %d", &m, &n, &p);
if (m <= 0 || n <= 0 || p <= 0)
{
printf("Dimensions must be positive.\n");
}
} while (m <= 0 || n <= 0 || p <= 0);
int A[m][n];
int B[n][p];
int C[n][p];
int BC[n][p];
int AB[m][p];
int AC[m][p];
int LHS[m][p];
int RHS[m][p];
printf("\nEnter elements of A (%d x %d):\n", m, n);
readMatrix(m, n, A);
printf("\nEnter elements of B (%d x %d):\n", n, p);
readMatrix(n, p, B);
printf("\nEnter elements of C (%d x %d):\n", n, p);
readMatrix(n, p, C);
/* B + C */
addMatrix(n, p, B, C, BC);
/* LHS = A(B+C) */
```

```
multiplyMatrix(m, n, p, A, BC, LHS);
/* AB and AC */
multiplyMatrix(m, n, p, A, B, AB);
multiplyMatrix(m, n, p, A, C, AC);
/* RHS = AB + AC */
addMatrix(m, p, AB, AC, RHS);
printf("\nLHS = A * (B + C):\n");
printMatrix(m, p, LHS);
printf("\nRHS = (A * B) + (A * C):\n");
printMatrix(m, p, RHS);
if (compareMatrix(m, p, LHS, RHS))
{
printf("\nLeft distributive property is VERIFIED.
\n");
printf("A*(B+C) = (A*B) + (A*C)\n");
}
else
{
printf("\nLeft distributive property is NOT verified.
\n");
}
}
/* Verify (A+B)C = AC + BC */
void rightDistributive(void)
{
int m, n, p;
do
{
printf("\nEnter m, n and p: ");
scanf("%d %d %d", &m, &n, &p);
if (m <= 0 || n <= 0 || p <= 0)
{
printf("Dimensions must be positive.\n");
}
} while (m <= 0 || n <= 0 || p <= 0);
```

```
int A[m][n];
int B[m][n];
int C[n][p];
int AB[m][n];
int AC[m][p];
int BC[m][p];
int LHS[m][p];
int RHS[m][p];
printf("\nEnter elements of A (%d x %d):\n", m, n);
readMatrix(m, n, A);
printf("\nEnter elements of B (%d x %d):\n", m, n);
readMatrix(m, n, B);
printf("\nEnter elements of C (%d x %d):\n", n, p);
readMatrix(n, p, C);
/* A + B */
addMatrix(m, n, A, B, AB);
/* LHS = (A+B)C */
multiplyMatrix(m, n, p, AB, C, LHS);
/* AC and BC */
multiplyMatrix(m, n, p, A, C, AC);
multiplyMatrix(m, n, p, B, C, BC);
/* RHS = AC + BC */
addMatrix(m, p, AC, BC, RHS);
printf("\nLHS = (A + B) * C:\n");
printMatrix(m, p, LHS);
printf("\nRHS = (A * C) + (B * C):\n");
printMatrix(m, p, RHS);
if (compareMatrix(m, p, LHS, RHS))
{
printf("\nRight distributive property is VERIFIED.
\n");
printf("(A+B)*C = (A*C) + (B*C)\n");
```

```
}
else
{
verified.\n");
}
printf("\nRight distributive property is NOT
}
```

3. Test-Case Table

Test Case	Choice / Input	Expected Result	Result
1	Choice 1, `m=2,n=2,p=2`	LHS = RHS; left distributive property verified	Pass
2	Choice 2, `m=2,n=2,p=2`	LHS = RHS; right distributive property verified	Pass
3	Choice 1, `m=1,n=1,p=1`	Property verified for boundary dimensions	Pass
4	`m=0,n=2,p=2`	Error: dimensions must be positive	Pass
5	`m=-1,n=2,p=3`	Error: dimensions must be positive	Pass
6	Choice `5`	Invalid choice message	Pass
7	Choice `3`	Program terminates	Pass